

Azure Master Data Engineer

6-Month Exhaustive Training & Certification Program

Advanced Architecture, Hands-on Labs, and MLOps

Abstract

This 24-week program is designed to transform you from a practitioner to a Master Data Engineer on Microsoft Azure. It progressively builds from core ADLS Gen2 data lakes to advanced Microsoft Purview governance, integrating daily theory, weekly service-specific mini-projects, architectural use case analyses, and massive monthly Capstone projects. A tracking timetable is included at the end.

Contents

1 Milestone 1: Core Data Infrastructure & Storage (Month 1)	2
2 Milestone 2: Data Warehousing & Analytics (Month 2)	6
3 Milestone 3: Batch Processing & ETL Orchestration (Month 3)	10
4 Milestone 4: Streaming & Real-Time Analytics (Month 4)	16
5 Milestone 5: Machine Learning Operationalization (Month 5)	22
6 Milestone 6: Security, Governance & Exam Prep (Month 6)	26
7 Appendix A: Glossary	31
8 Appendix B: Assessment & Grading Model	32
9 Appendix C: Interview Preparation	33
10 Appendix D: Self-Check Answer Key	33
11 Appendix E: Datasets & Project Data	37
12 Appendix F: Portfolio & Presentation Guide	38
13 6-Month Progress Tracking Timetable	39

Prerequisites & How to Use This Syllabus

- **Prerequisite skills:** working knowledge of Python and SQL, Git fundamentals (clone/branch/commit/Push), Windows/Linux command-line basics, Docker fundamentals (images, containers, registries), and a free Azure subscription.
- **Weekly cadence:** ~10–12 hours per week. Monday–Wednesday cover theory, Thursday is a hands-on project, Friday is an architecture use case.
- **Deliverable expectation:** every monthly capstone must produce a git repository, an architecture diagram, a monthly cost estimate, and a security review checklist, per the Acceptance Criteria boxes after each capstone.
- **Self-assessment:** the 24-week tracking timetable at the end doubles as your assessment checklist — check off a week only when its project or capstone is complete and documented. Each week also ends with a Self-check; answers are in Appendix D.

Environment setup checklist

- Install Python 3.x, Git, Docker Desktop, and VS Code (with the Python and Azure extensions).
- Install the platform CLIs: Azure CLI (az), AzCopy, Databricks CLI, and Azure Functions Core Tools.
- Create a free Azure account (12 months of free services plus a \$200 credit) and enable a cost alert from day one.
- Clone or create a course repo scaffold: `data/`, `src/`, `tests/`, `infra/`, `docs/`.
- Configure credentials via environment variables, Azure CLI login, and Managed Identities — never hardcode secrets in code or notebooks.

1 Milestone 1: Core Data Infrastructure & Storage (Month 1)

Objective: Master the foundational storage options, relational databases, and NoSQL.

Learning outcomes: deploy and tier ADLS Gen2 storage, configure geo-redundant Azure SQL, design Cosmos DB partition keys and consistency, benchmark Redis caching, estimate storage costs.

Week 1: ADLS Gen2 & Storage Foundations

Outcomes: deploy ADLS Gen2 with a hierarchical namespace, author lifecycle rules that tier blobs to Archive, compare access-tier costs for a given access pattern, restore a soft-deleted blob.

- **Monday:** Azure Storage Architecture, Blob Storage vs. Azure Data Lake Storage (ADLS) Gen2.
- **Tuesday:** Access Tiers (Hot, Cool, Cold, Archive), Lifecycle Management, and Soft Delete.
- **Wednesday:** Azure Data Box, AzCopy, and Azure Storage Explorer.
- **Thursday: Hands-on Project:** Deploy an ADLS Gen2 account via Azure CLI, configure a 30-day lifecycle transition to Archive tier, and set up a hierarchical namespace.

- **Friday: Use Case & Architecture:** Design a multi-region (Geo-Zone-Redundant), compliant disaster recovery storage architecture for a healthcare provider processing 10PB of X-ray images.

Resources: Azure Blob Storage documentation (<https://learn.microsoft.com/azure/storage/blobs/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>)

Self-check:

- How does ADLS Gen2 differ from standard Blob Storage?
- When would you choose the Archive tier, and what is the trade-off?
- What does a lifecycle management rule automate?
- How do soft delete and blob versioning protect against accidental deletion?
- Which tool moves 10 TB of on-prem files to ADLS, and when would you use Data Box instead?

Production Note: Enable soft delete and blob versioning on production ADLS accounts and rehearse recovery: restore an accidentally deleted container via soft-delete undelete, and recover an overwritten blob by promoting a previous version. Backup features you have never restored from are not a DR plan.

Week 2: Relational Databases (Azure SQL & PostgreSQL)

Outcomes: choose between DTU and vCore pricing for a workload, configure auto-failover groups, execute a point-in-time restore to a new database, measure failover RTO.

- **Monday:** Azure SQL Database: DTU vs. vCore models, Serverless, and Hyperscale.
- **Tuesday:** Azure Database for PostgreSQL: Flexible Server and High Availability (HA).
- **Wednesday:** Active Geo-Replication and Auto-failover groups.
- **Thursday: Hands-on Project:** Deploy an Azure SQL Database Serverless tier, create a geo-replica in another region, and simulate a regional failover. DR drill: perform a point-in-time restore of the database to a NEW database (e.g., 15 minutes back), verify row counts match the source at that timestamp, then drop the restored database to avoid charges.
- **Friday: Use Case & Architecture:** Architect a zero-downtime migration strategy from a monolithic on-premise SQL Server to Azure SQL Managed Instance.

Resources: Azure SQL Database documentation (<https://learn.microsoft.com/azure/azure-sql/>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-handbook>). **Self-**

check:

- DTU vs. vCore: which model gives you direct control over hardware?
- What does an auto-failover group provide beyond a single geo-replica?
- Why restore a point-in-time backup to a new database instead of in place?
- Why is Azure SQL Serverless cheap for intermittent workloads?
- How does active geo-replication differ from a read scale-out replica?

Week 3: High-Scale NoSQL (Azure Cosmos DB)

Outcomes: pick partition keys that spread RU load evenly, apply consistency levels per workload, consume the Change Feed from Python, diagnose and fix a hot partition.

- **Monday:** Cosmos DB core concepts: Request Units (RUs), Partitioning, and Global Distribution.
- **Tuesday:** Consistency Levels (Strong, Bounded Staleness, Session, Consistent Prefix, Eventual).
- **Wednesday:** APIs (Core SQL, MongoDB, Cassandra) and Cosmos DB Change Feed.
- **Thursday:** **Hands-on Project:** Create a Cosmos DB account (SQL API), design a single-container schema for a forum, write a Python script to insert items, and configure multi-region writes.
- **Friday:** **Use Case & Architecture:** Design an ingestion and serving layer for a gaming leaderboard supporting 1 million reads/writes per second globally with Session consistency.

Resources: Azure Cosmos DB documentation (<https://learn.microsoft.com/azure/cosmos-db/>); Cosmos DB Change Feed design patterns (<https://learn.microsoft.com/azure/cosmos-db/change-feed-d>)

Self-check:

- What is a Request Unit, and why do cross-partition queries cost more RUs?
- Which consistency level guarantees reads see the latest write?
- When is Session consistency a good default?
- What ordering does the Cosmos DB Change Feed guarantee?
- Why does a skewed partition key (e.g., one celebrity gamer) cause throttling?

Week 4: Hybrid Ingestion & Caching

Outcomes: implement cache-aside with Redis TTLs, compare Cosmos DB integrated cache with standalone Redis, front an ingestion API with APIM throttling, absorb a 10× traffic spike.

- **Monday:** Azure Cache for Redis for database acceleration.
- **Tuesday:** Cosmos DB Integrated Cache.
- **Wednesday:** Azure API Management (APIM) for data ingestion.
- **Thursday:** **Hands-on Project:** Deploy an Azure Cache for Redis instance. Write a Python script to cache query results from an Azure SQL Database to reduce load.
- **Friday:** **Use Case & Architecture:** Architect a high-throughput, low-latency data API for a mobile application that experiences extreme traffic spikes during live sports events.

Resources: Azure Cache for Redis documentation (<https://learn.microsoft.com/azure/azure-cache-for-redis/>); Azure API Management documentation (<https://learn.microsoft.com/azure/api-management/>).

Self-check:

- What is the cache-aside pattern and when do you use it?
- How does the Cosmos DB integrated cache differ from Azure Cache for Redis?

- Why put API Management in front of a data ingestion API?
- What happens to cached query results when the underlying data changes?
- How would you absorb a sudden 10× traffic spike on a Redis-backed API?

CAPSTONE PROJECT - Milestone 1: Build an E-Commerce Multi-Tier Data Backend. Deploy Azure SQL Database Serverless for user profiles/billing, Cosmos DB for active shopping carts and clickstream tracking, ADLS Gen2 for product images, and set up Cosmos DB Change Feed to push events to storage.

Acceptance Criteria

- End-to-end flow.
- Architecture diagram.
- At least one demo.
- Monitor performance.
- Monitor costs.
- Security review.

Milestone 1 Capstone: Reference Solution Sketch

- **Architecture (one line):** Azure SQL Serverless for billing + Cosmos DB (Session consistency, partitioned by /customerId) for carts/clickstream + ADLS Gen2 for images/archives, with Change Feed → ADLS event propagation — each tier matched to its access pattern instead of one store forced to do everything.
- **Steps, in order:** (1) provision all services via Bicep/CLI with Managed Identities; (2) model the cart container around /customerId; (3) enable Change Feed and deploy the processor; (4) land events append-only in ADLS; (5) add diagnostic settings and cost alerts; (6) write README + diagram.
- Hardest part — a minimal Change Feed processor that is safe under replay:

```
from azure.cosmos import CosmosClient
from azure.identity import DefaultAzureCredential

def handle_changes(changes):
    for doc in changes:
        path = f"clickstream/{doc['customerId']}/{doc['id']}.json"
        adls.upload(path, json.dumps(doc), overwrite=True) # idempotent

container = CosmosClient(URL, DefaultAzureCredential()) \
    .get_container_client("shop", "carts")
for change in container.query_items_change_feed(is_start_from_beginning=True):
    handle_changes([change])
```

- **Pitfall:** a skewed partition key (one whale customer) creates a hot partition and 429 throttling — pick high-cardinality keys.
- **Pitfall:** overwriting on Change Feed re-delivery corrupts state unless the sink write is idempotent (same id → same blob).
- **Pitfall:** leaving storage public-access enabled — disable it and use Managed Identity from day one.

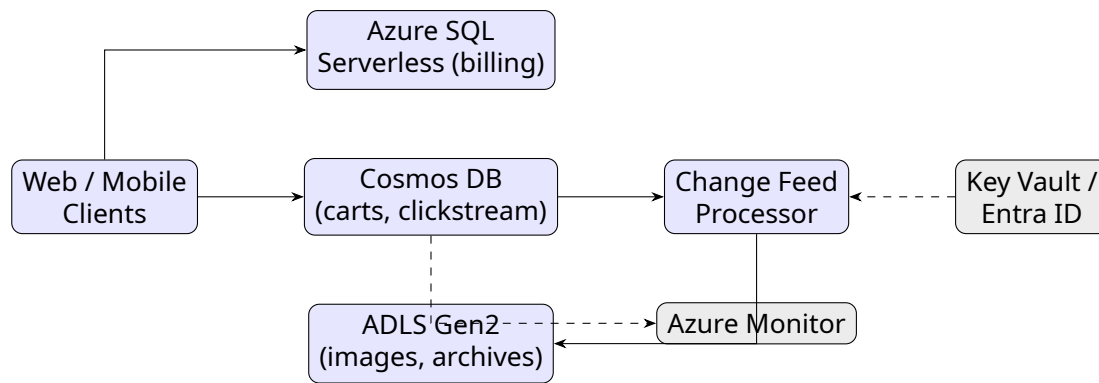


Figure 1: Milestone 1 reference architecture: multi-tier backend (ADLS / Azure SQL / Cosmos DB)

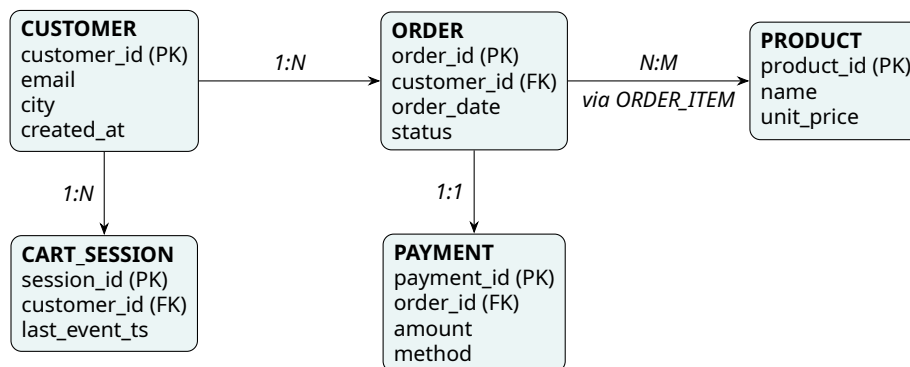


Figure 2: Capstone data model: e-commerce multi-tier backend

2 Milestone 2: Data Warehousing & Analytics (Month 2)

Objective: Deep dive into Azure Synapse Analytics, Lakehouses, and Purview.

Learning outcomes: choose and tune Synapse distribution keys, benchmark columnstore queries, build Lakehouse views over Delta Lake, catalog and classify data in Purview, control warehouse cost.

Week 5: Azure Synapse Analytics (Dedicated SQL Pools)

Outcomes: bulk-load Parquet via COPY with Managed Identity, declare fact-table grain, implement SCD Type 2 dimensions with MERGE, choose hash vs. replicate distributions.

- **Monday:** Synapse Architecture (formerly SQL DW), MPP architecture, and Data Warehouse Units (cDWUs). Kimball dimensional modeling: declaring the grain, fact types (transaction, periodic snapshot, accumulating snapshot), conformed dimensions, and SCD Type 1 vs. Type 2.
- **Tuesday:** Distribution Types (Hash, Round-Robin, Replicate).
- **Wednesday:** Data ingestion: PolyBase vs. COPY statement.
- **Thursday: Hands-on Project:** Provision a Synapse Dedicated SQL Pool. Load 10 million rows of synthetic Parquet data from ADLS Gen2 using the COPY command with Managed Identity. Model the data as a star schema (declare the grain first) and apply the SCD Type 2 MERGE below to keep customer history.

```
-- SCD Type 2 upsert on a Synapse Dedicated SQL Pool
MERGE INTO dim_customer AS tgt
USING stg_customer AS src
  ON tgt.customer_id = src.customer_id AND tgt.is_current = 1
WHEN MATCHED AND (tgt.email <> src.email OR tgt.city <> src.city) THEN
  UPDATE SET tgt.valid_to = CAST(GETDATE() AS date), tgt.is_current = 0
WHEN NOT MATCHED THEN
  INSERT (customer_id, email, city, valid_from, valid_to, is_current)
  VALUES (src.customer_id, src.email, src.city,
    CAST(GETDATE() AS date), '9999-12-31', 1);

-- insert new versions for the rows just closed by the MERGE
INSERT INTO dim_customer (customer_id, email, city, valid_from, valid_to, is_current)
SELECT src.customer_id, src.email, src.city,
  CAST(GETDATE() AS date), '9999-12-31', 1
FROM stg_customer AS src
JOIN dim_customer AS tgt
  ON tgt.customer_id = src.customer_id AND tgt.is_current = 1
WHERE tgt.email <> src.email OR tgt.city <> src.city;
```

- **Friday: Use Case & Architecture:** Estimate the monthly cost and recommend an architecture (Dedicated vs. Serverless) for a BI workload heavily utilized only during end-of-month reporting.

Resources: Azure Synapse Analytics documentation (<https://learn.microsoft.com/azure/synapse-analytics/synapse-dedicated-sql-pool-best-practices>). **Self-check:**

- Why must you declare the grain before designing a fact table?
- Which fact type models a process with milestones (order → ship → deliver)?
- When is SCD Type 2 required instead of Type 1?
- Why use conformed dimensions across multiple data marts?
- When does Serverless SQL beat a Dedicated pool on cost?

Production Note: *Pause or scale down Dedicated SQL Pools outside working hours (or use serverless for spiky workloads); an idle DW1000c pool can cost thousands of dollars per month. Set an Azure Monitor cost alert as a guardrail.*

Azure compute cost comparison (warehousing):

Option	Billing model	Best for	Watch-outs
Synapse Dedicated SQL Pool	cDWU per hour; pauseable	Predictable, high-concurrency warehouse workloads	Idle pools still bill; pause/resize off-hours
Synapse Serverless SQL	Per TB of data scanned	Ad-hoc exploration, logical DW over the lake	Cost scales with bytes scanned; prune partitions, use Parquet
Microsoft Fabric capacity (F SKU)	Reserved capacity units, per-second billing, pauseable	Unified lakehouse + Power BI on one capacity	Burst-then-throttle when oversubscribed; size the SKU
Azure Database SQL	vCore/hour or DTU; serverless option	OLTP systems and small departmental marts	Not MPP; scale ceiling below Synapse for large analytics

Week 6: Synapse Optimization & Performance

Outcomes: benchmark distribution types on JOIN queries, apply clustered columnstore and partitioning correctly, configure workload groups and classifiers, decide when a materialized view pays off.

- **Monday:** Indexing (Clustered Columnstore, Heap, Non-clustered).
- **Tuesday:** Partitioning strategies in Synapse Dedicated Pools.
- **Wednesday:** Workload Management (Workload Groups and Classifiers) and Materialized Views.
- **Thursday: Hands-on Project:** Create three identical tables in Synapse using different Distribution Types. Benchmark the execution time of a complex JOIN query on each table.
- **Friday: Use Case & Architecture:** Re-architect a poorly performing multi-petabyte analytics table that is causing CPU bottlenecks during complex daily dashboard aggregations.

Resources: Synapse workload management documentation (<https://learn.microsoft.com/azure/synapse-analytics/sql-data-warehouse/sql-data-warehouse-workload-management>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-handbook>). **Self-check:**

- Why is clustered columnstore the default for large fact tables?
- What problem does hash distribution solve for large JOINS?
- When is a replicated table the right distribution choice?
- What do workload classifiers map incoming queries to?
- When does a materialized view beat adding an index?

Week 7: Data Lakehouse & Synapse Serverless SQL

Outcomes: query ADLS files with serverless OPENROWSET, materialize results with CETAS, build a logical data warehouse of views, cut per-TB scan cost with Parquet and partition pruning.

- **Monday:** Synapse Serverless SQL Pools: Querying directly over ADLS Gen2 without provisioning compute.
- **Tuesday:** Logical Data Warehouse pattern (Views and External Tables).
- **Wednesday:** Delta Lake format concepts (ACID transactions on data lakes).
- **Thursday: Hands-on Project:** Write a Serverless SQL Pool query to read raw CSVs in ADLS Gen2, transform them on the fly, and save the output as Parquet (CETAS) back to the lake.
- **Friday: Use Case & Architecture:** Design a cost-effective "Lakehouse" architecture for a media company wanting to store 50PB of logs in ADLS Gen2 while querying them alongside core business data.

Resources: Synapse Serverless SQL documentation (<https://learn.microsoft.com/azure/synapse-analytics/sql/on-demand-workspace-overview>); Delta Lake project (<https://github.com/delta-io/delta>).

Self-check:

- How is Serverless SQL billed, and how do you cut that cost?
- What is the logical data warehouse pattern?

- What does Delta Lake add to plain files on ADLS Gen2?
- What does CETAS do?
- Why store curated data as Delta/Parquet instead of CSV?

Week 8: Data Governance & Microsoft Purview

Outcomes: register and scan sources in Purview, auto-classify PII columns, trace lineage from lake to report, share datasets across tenants with Azure Data Share.

- **Monday:** Microsoft Purview: Centralized data map and cataloging.
- **Tuesday:** Data Lineage and automated classification of sensitive data.
- **Wednesday:** Azure Data Share for sharing data across Azure tenants.
- **Thursday: Hands-on Project:** Configure Microsoft Purview over an ADLS Gen2 bucket and Synapse workspace. Run a scan to automatically classify "PII_Email" columns and trace lineage.
- **Friday: Use Case & Architecture:** Architect a Data Clean Room for an ad agency to merge their CRM data with third-party advertiser data using Azure Data Share without exposing raw PII.

Resources: Microsoft Purview documentation (<https://learn.microsoft.com/azure/purview/>); Azure Data Share documentation (<https://learn.microsoft.com/azure/data-share/>). **Self-check:**

- What does a Purview scan automatically discover?
- How does lineage help impact analysis before a schema change?
- Why use Azure Data Share instead of copying files to a partner?
- What is a data clean room, and what must it never expose?
- How would you automatically tag PII_Email columns across a whole lake?

CAPSTONE PROJECT - Milestone 2: Enterprise Data Lakehouse Build. Ingest 5 years of retail data into ADLS Gen2 using Delta Lake format. Use Microsoft Purview to catalog and classify the data. Create Synapse Serverless SQL Pool Views to serve executive dashboards logically without duplicating data into a dedicated warehouse.

Acceptance Criteria

- End-to-end data pipeline
- Architecture layer
- At least one table
- Monitored exposure
- Monitored decision
- Security

Milestone 2 Capstone: Reference Solution Sketch

- **Architecture (one line):** ADLS Gen2 bronze/silver/gold in Delta Lake format, transformed with Databricks, served as Synapse Serverless SQL views to Power BI, cataloged by Purview — serverless serving avoids paying for an idle dedicated warehouse.
- **Steps, in order:** (1) land raw retail files in bronze; (2) clean/dedupe into silver Delta tables; (3) model gold as a star schema (grain declared first, SCD2 customers); (4) run OPTIMIZE + VACUUM on Delta tables; (5) register and scan in Purview with PII classifications; (6) expose gold via serverless views; (7) connect Power BI with DirectQuery.
- Hardest part — a serverless view over Delta gold data:

```
CREATE VIEW gold.vw_sales_by_region AS
SELECT s.region, d.year, SUM(f.net_amount) AS revenue
FROM OPENROWSET(
    BULK 'https://<adls>.dfs.core.windows.net/gold/fact_sales/',
    FORMAT = 'DELTA') AS f
JOIN gold.dim_store s ON f.store_key = s.store_key
JOIN gold.dim_date d ON f.date_key = d.date_key
GROUP BY s.region, d.year;
```

- **Pitfall:** leaving gold as CSV — serverless bills per TB scanned; Parquet/Delta plus partition pruning cuts cost 10×+.
- **Pitfall:** the small-file problem from frequent appends — schedule OPTIMIZE or dashboards slow down over time.
- **Pitfall:** scanning with Purview only after dashboards ship — classify PII before any serving view exposes it.

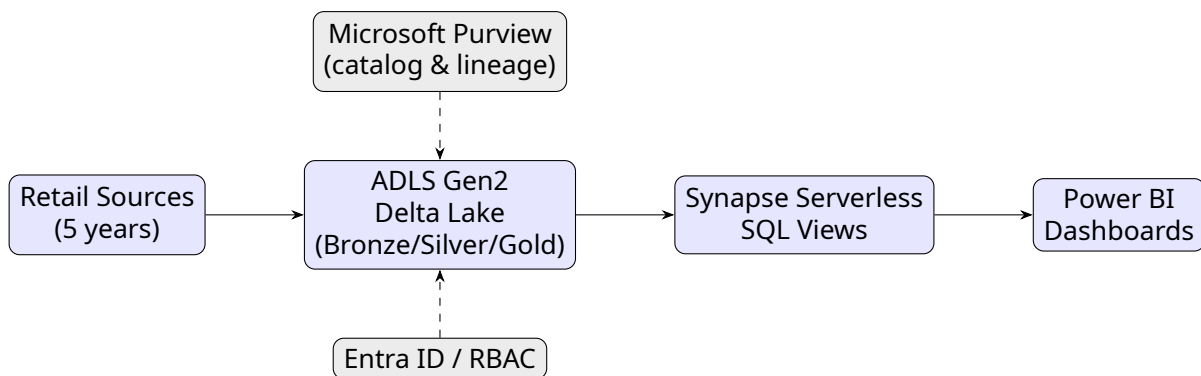


Figure 3: Milestone 2 reference architecture: Lakehouse (ADLS → Synapse Serverless → Purview → Power BI)

3 Milestone 3: Batch Processing & ETL Orchestration (Month 3)

Objective: Master Azure Databricks, Azure Data Factory (ADF), and Data Migration.

Learning outcomes: author PySpark jobs on Databricks, build Mapping Data Flows, implement CDC pipelines with DMS, orchestrate event-driven batch ELT in ADF, diagnose pipeline failures.

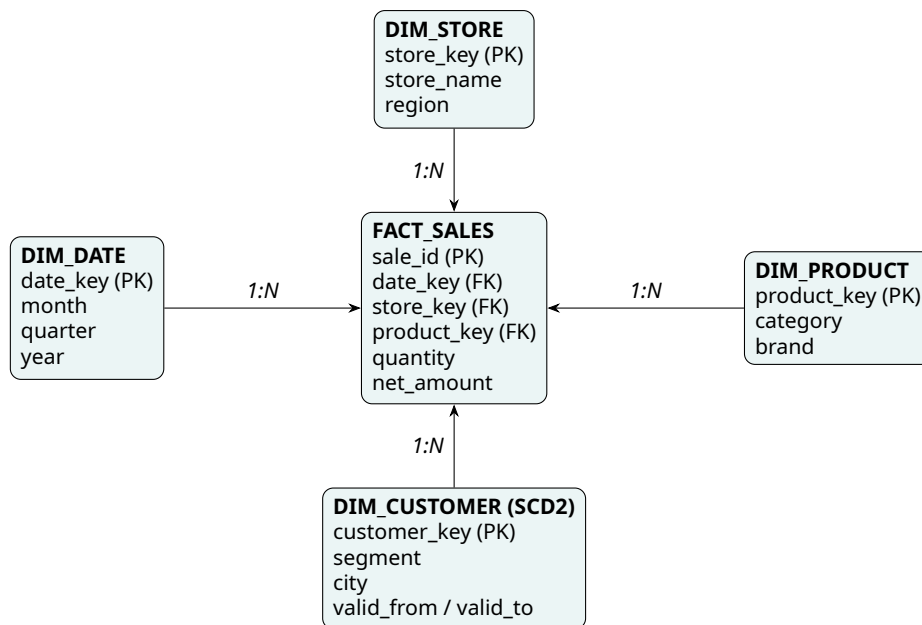


Figure 4: Capstone data model: retail lakehouse star schema

Week 9: Azure Databricks (Managed Spark)

Outcomes: author PySpark notebooks against mounted ADLS, explain and tune Spark partitions and shuffles, build Delta Live Tables pipelines, choose job clusters over all-purpose clusters for production.

- **Monday:** Databricks Workspace, Clusters (All-Purpose vs. Job), and DBFS.
- **Tuesday:** Apache Spark internals on Databricks (DataFrames, RDDs, Partitioning).
- **Wednesday:** Databricks Delta Lake and Delta Live Tables.
- **Thursday: Hands-on Project:** Deploy an Azure Databricks workspace. Submit a PySpark notebook that reads data from ADLS Gen2, performs a distributed aggregation, and writes to a Delta table.
- **Friday: Use Case & Architecture:** Design a migration strategy for a 100-node on-premise Hadoop cluster, prioritizing cost reduction and moving the processing engine to Azure Databricks.

Resources: Azure Databricks documentation (<https://learn.microsoft.com/azure/databricks/>); Apache Spark project (<https://github.com/apache/spark>). **Self-check:**

- Why use job clusters instead of all-purpose clusters in production?
- What does a Spark partition control during a shuffle?
- How do Delta Live Tables differ from plain Delta tables?
- When is caching a DataFrame in Databricks worthwhile?
- What is DBFS, and why should production data not live there?

Production Note: Use job clusters (not all-purpose clusters) for production workloads — they terminate on completion and cost 2–3× less per DBU. Set cluster auto-termination and enforce it with Databricks cluster policies.

Week 10: Visual ETL (Azure Data Factory)

Outcomes: build ADF pipelines from linked services and datasets, author a Mapping Data Flow join/clean/load, select the right Integration Runtime, parameterize pipelines for dev/test/prod promotion.

- **Monday:** ADF Concepts: Linked Services, Datasets, Pipelines, and Activities.
- **Tuesday:** Mapping Data Flows (Visual, code-free data transformations backed by Spark).
- **Wednesday:** Integration Runtimes (Azure, Self-Hosted, Azure-SSIS).
- **Thursday: Hands-on Project:** Build a visual pipeline in ADF using Mapping Data Flows that joins two datasets from ADLS Gen2, drops nulls, transforms timestamps, and loads the clean data into Synapse.

```
{
  "name": "pl_join_clean_load",
  "properties": {
    "activities": [
      {
        "name": "JoinCleanDataFlow",
        "type": "ExecuteDataFlow",
        "typeProperties": {
          "dataflow": { "referenceName": "df_join_orders_customers" },
          "compute": { "coreCount": 8, "computeType": "General" },
          "staging": {
            "linkedService": { "referenceName": "ls_adls_staging" },
            "folderPath": "adf/staging"
          }
        },
        "onSuccess": [{ "activity": "AuditRowCounts" }],
        "onFailure": [{ "activity": "NotifyFailureLogicApp" }]
      }
    ]
  }
}
```

- **Friday: Use Case & Architecture:** Evaluate Mapping Data Flows in ADF vs. Databricks Notebooks for a team of business analysts who need to build daily reporting pipelines without writing code.

Resources: Azure Data Factory documentation (<https://learn.microsoft.com/azure/data-factory/>); Data Flow expression language reference (<https://learn.microsoft.com/azure/data-factory/data-transformation-functions>). **Self-check:**

- What is the difference between a Linked Service and a Dataset in ADF?
- When do Mapping Data Flows beat hand-written Spark code?
- Why would you need a Self-Hosted Integration Runtime?
- What happens in ADF when no onFailure path is defined on an activity?
- How do you parameterize a pipeline for dev/test/prod promotion?

Week 11: Change Data Capture (CDC), Migration & Data Contracts

Outcomes: run a DMS migration with CDC enabled, validate versioned JSON data contracts before landing, route contract violations to quarantine, monitor and alert on replication lag.

- **Monday:** Azure Database Migration Service (DMS) architecture.
- **Tuesday:** CDC features in Azure SQL and Cosmos DB Change Feed.
- **Wednesday:** Event-driven data pipelines using Azure Event Grid. Data contracts: JSON schema validation of incoming payloads in ADF (Data Flow derived-column/assert patterns) or Databricks (schema enforcement, `spark.read.schema(...)`) before landing in ADLS; contract versioning and compatibility (breaking vs. non-breaking changes); producer vs. consumer ownership of the contract.
- **Thursday: Hands-on Project:** Configure an Azure DMS task with CDC enabled to stream ongoing database changes from an Azure SQL Database to an ADLS Gen2 Data Lake. Add a contract gate: validate each payload against a versioned JSON schema in a Databricks notebook (or ADF Data Flow) before landing, and route contract-violating records to a quarantine container.
- **Friday: Use Case & Architecture:** Architect a near-real-time synchronization pipeline from an on-premise Oracle database to Azure Synapse Analytics for live BI reporting.

Resources: Azure Database Migration Service documentation (<https://learn.microsoft.com/azure/dms/>); Azure Event Grid documentation (<https://learn.microsoft.com/azure/event-grid/>).

Self-check:

- Why validate JSON schemas before landing data in ADLS?
- Is adding an optional field a breaking or non-breaking contract change?
- Who owns a data contract — the producer or the consumer?
- What does contract versioning let a producer do safely?
- How does CDC differ from a nightly full extract?

Production Note: Monitor CDC replication latency with Azure Monitor metrics on the DMS task (e.g., CDC latency / commit lag); alert when lag exceeds your SLA (e.g., 5 minutes). Stalled CDC quietly breaks downstream freshness — treat lag as a first-class SLO with a runbook entry.

Week 12: Pipeline Orchestration (ADF & Apache Airflow)

Outcomes: compose control-flow pipelines (ForEach, Until, Execute Pipeline), trigger pipelines from Event Grid, route failures to Logic Apps, deploy DAGs on Managed Airflow in ADF.

- **Monday:** Advanced ADF Control Flow: ForEach, If Condition, Until, Execute Pipeline.
- **Tuesday:** Managed Airflow in Azure Data Factory (formerly Azure Workflow parameters).
- **Wednesday:** Azure Logic Apps for business process automation and notifications.
- **Thursday: Hands-on Project:** Create an ADF pipeline that: 1. Waits for an Event Grid trigger. 2. Triggers a Databricks Notebook. 3. Calls a Logic App to send a Teams message on failure.
- **Friday: Use Case & Architecture:** Design a resilient, self-healing orchestration layer for a daily banking reconciliation process spanning 15 dependent systems using ADF and Logic Apps.

Resources: ADF pipeline execution and triggers (<https://learn.microsoft.com/azure/data-factory/concepts-pipeline-execution-triggers>); Apache Airflow project (<https://github.com/apache/airflow>). **Self-check:**

- When would you use an Until activity instead of a ForEach in ADF?
- What problem does Managed Airflow in ADF solve?
- Why route failure notifications through a Logic App instead of plain email?
- What makes an orchestration layer “self-healing”?
- How do Event Grid triggers change pipeline scheduling?

Production Note: Create an Azure Monitor alert rule on ADF pipeline-run failures: send the factory's PipelineRuns to a Log Analytics workspace via diagnostic settings, then define a log alert (severity 1, 5-minute evaluation window) wired to an Action Group that notifies on-call via Teams/email. Pair every alert with a runbook describing triage steps.

CAPSTONE PROJECT - Milestone 3: Automated Batch ELT Pipeline. Use Azure Event Grid to trigger an ADF pipeline upon file upload to ADLS Gen2. The pipeline triggers a Databricks notebook to perform complex data quality checks and ML feature engineering, and uses the ADF Copy Activity to bulk load the output into a Synapse Dedicated SQL Pool.

Acceptance Criteria

- End-to-end flow of a file
- Architecture of Data
- At least one demo
- Monitoring and screening
- Monitoring and notification
- Security and compliance

Milestone 3 Capstone: Reference Solution Sketch

- **Architecture (one line):** Event Grid BlobCreated trigger → ADF pipeline → Databricks notebook (contract validation + DQ + feature engineering, quarantining bad files) → ADF COPY into a hash-distributed Synapse Dedicated table — event-driven so no polling cost.
- **Steps, in order:** (1) create the Event Grid system topic on the storage account; (2) build the ADF pipeline with a blob-event trigger passing folderPath/fileName as parameters; (3) write the Databricks notebook: schema check → quarantine or transform → write curated Delta; (4) add the COPY activity with Managed Identity and hash distribution on the join key; (5) wire a Logic App failure alert; (6) test the happy path and a poison file.
- Hardest part — the blob-event trigger definition:

```
{
  "name": "trg_blob_created",
  "properties": {
    "type": "BlobEventsTrigger",
    "typeProperties": {
```

```

    "blobPathBeginsWith": "/raw/incoming/",
    "ignoreEmptyBlobs": true,
    "events": ["Microsoft.Storage.BlobCreated"]
  }
}
}

```

- **Pitfall:** the trigger fires on partial uploads — write to a temp path and rename/commit so only complete files land in /raw/incoming/.
- **Pitfall:** re-running the pipeline double-loads rows — make the notebook idempotent (merge on file ID, not blind append).
- **Pitfall:** round-robin distribution on the target table shuffles every JOIN — hash-distribute on the dominant join key.

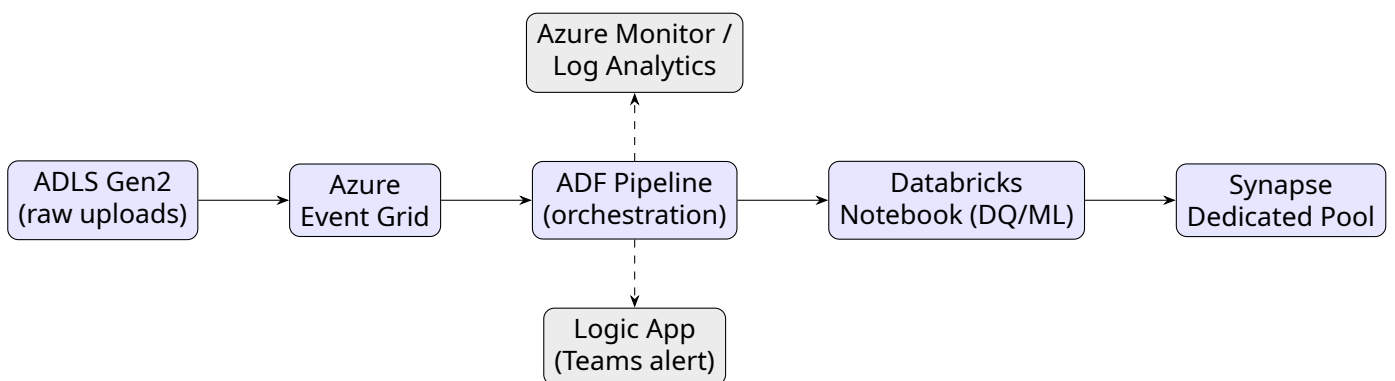


Figure 5: Milestone 3 reference architecture: batch ELT (Event Grid → ADF → Databricks → Synapse)

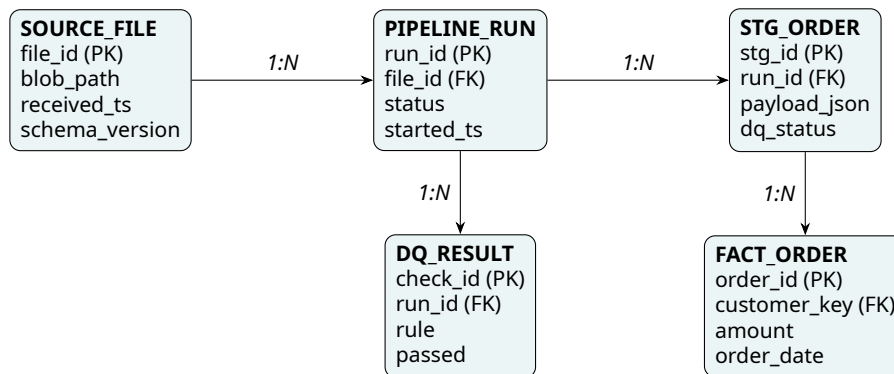


Figure 6: Capstone data model: automated batch ELT pipeline

4 Milestone 4: Streaming & Real-Time Analytics (Month 4)

Objective: Conquer unbounded data, Event Hubs, Stream Analytics, and Power BI.

Learning outcomes: design partition strategies in Event Hubs, implement idempotent producers and checkpointed consumers, write ASA windowed queries, build Spark Structured Streaming jobs, serve real-time dashboards in Power BI.

Week 13: Message Queuing & Event Ingestion

Outcomes: choose between Service Bus and Event Hubs, design a partition strategy and consumer groups, write an idempotent producer with dedupe keys, enable Event Hubs Capture to ADLS.

- **Monday:** Decoupling with Azure Service Bus (Queues/Topics).
- **Tuesday:** Azure Event Hubs: Partitions, Consumer Groups, and Throughput Units.
- **Wednesday:** Event Hubs Capture (automatic delivery to ADLS Gen2) and Kafka compatibility.
- **Thursday: Hands-on Project:** Create an Azure Event Hub. Write a Python producer script to stream mock IoT temperature sensor data, and configure Event Hubs Capture to save batches to ADLS Gen2.

```
import asyncio, json, random, time
from azure.eventhub.aio import EventHubProducerClient
from azure.eventhub import EventData

CONN_STR = "Endpoint=sb://<ns>.servicebus.windows.net/;..." # from Key Vault
HUB_NAME = "iot-telemetry"

async def run():
    producer = EventHubProducerClient.from_connection_string(
        conn_str=CONN_STR, eventhub_name=HUB_NAME)
    async with producer:
        while True:
            batch = await producer.create_batch()
            for sensor_id in range(1, 6):
                event = {
                    "deviceId": f"sensor-{sensor_id}",
                    "eventId": f"{sensor_id}-{int(time.time())}", # dedupe key
                    "tempC": round(random.uniform(-20, 45), 2),
                    "ts": time.time(),
                }
                batch.add(EventData(json.dumps(event)))
            await producer.send_batch(batch)
            await asyncio.sleep(1)

asyncio.run(run())
```

- **Friday: Use Case & Architecture:** Architect a global telemetry ingestion point for 5 million mobile games, ensuring high throughput, partitioning by user ID, and low-latency processing.

Resources: Azure Event Hubs documentation (<https://learn.microsoft.com/azure/event-hubs/>); Apache Kafka project (<https://github.com/apache/kafka>). **Self-check:**

- When do you choose Service Bus over Event Hubs?

- Why does the partition count cap consumer parallelism in Event Hubs?
- What does Event Hubs Capture automate?
- Why attach a unique `eventId` to every message?
- What is a consumer group, and why create more than one?

Production Note: *Make consumers idempotent: retries and Event Hubs replay can deliver the same event more than once (at-least-once semantics). Attach a unique `eventId` per message, dedupe on sink upserts, and persist checkpoints to Blob Storage so consumers resume from the last committed offset — never from in-memory state.*

Week 14: Real-Time Processing (Azure Stream Analytics)

Outcomes: write ASA queries with tumbling, hopping, and sliding windows, configure late-arrival and out-of-order policies, sink aggregates to Azure SQL, alert on watermark delay.

- **Monday:** Azure Stream Analytics (ASA) architecture and SQL-like query language.
- **Tuesday:** Windowing functions: Tumbling, Hopping, Sliding, and Session windows.
- **Wednesday:** Handling late-arriving data and out-of-order events in ASA.
- **Thursday: Hands-on Project:** Write an ASA job that reads from Event Hubs, groups user clicks into 5-minute Tumbling Windows, counts the clicks, and writes the output to Azure SQL Database.
- **Friday: Use Case & Architecture:** Troubleshoot a scenario where a streaming dashboard is showing inaccurate daily aggregations due to mobile devices going offline and uploading data late.

Resources: Azure Stream Analytics documentation (<https://learn.microsoft.com/azure/stream-analytics>)
 ASA query reference — windowing (<https://learn.microsoft.com/stream-analytics-query/windowing-azure-stream-analytics>). **Self-check:**

- Contrast tumbling and hopping windows with an example.
- What does the late-arrival tolerance window control in ASA?
- What happens to events outside the out-of-order tolerance?
- Why does a silent input stall look identical to zero traffic on a dashboard?
- How would you fix daily aggregates skewed by late-uploading mobile devices?

Production Note: *Explicitly configure late-arrival and out-of-order tolerance windows in ASA and decide a policy for what happens to events outside the window (adjust vs. drop). Set a watermark alert: a silent input stall (no events arriving) otherwise looks identical to "zero traffic" on dashboards.*

Week 15: Advanced Streaming (Spark Structured Streaming)

Outcomes: consume Event Hubs via the Kafka endpoint in Spark, manage streaming state with watermarks and the RocksDB state-store provider, join two streams with bounded state retention, and deliver end-to-end exactly-once via checkpointing plus an idempotent Delta merge sink.

- **Monday:** Apache Spark Structured Streaming on Azure Databricks. State management: the state store (default in-memory vs. the RocksDB provider for large state), checkpoint locations holding offsets plus state, watermark-driven state eviction, and why unbounded state kills streaming jobs — state that is never evicted grows until executors run out of memory and the job fails.
- **Tuesday:** Micro-batch processing vs. Continuous Processing. End-to-end exactly-once: checkpointing plus idempotent sinks (Delta Lake merge as an exactly-once sink) vs. at-least-once delivery with downstream deduplication.
- **Wednesday:** Integrating Event Hubs with Databricks using the Kafka API. Stream-stream joins: inner and left joins with watermark constraints, and how the watermark plus join time-range condition bound state retention on both sides.

```
from pyspark.sql import functions as F

# Watermarks on both sides bound how long join state is retained
impressions = impressions.withWatermark("impression_ts", "10 minutes")
clicks = clicks.withWatermark("click_ts", "20 minutes")

joined = impressions.join(
    clicks,
    F.expr("""click_ad_id = impression_ad_id
            AND click_ts BETWEEN impression_ts
            AND impression_ts + INTERVAL 30 MINUTES"""),
    "leftOuter",
)

(joined.writeStream.format("delta")
 .outputMode("append")
 .option("checkpointLocation",
        "abfss://checkpoints@<adls>.dfs.core.windows.net/ad-join")
 .toTable("ad_attribution"))
```

- **Thursday: Hands-on Project:** Write a Databricks notebook using Spark Structured Streaming to consume data from Event Hubs, aggregate by window, and append the results into a Delta Lake table.

```
from pyspark.sql import functions as F

# Event Hubs exposes a Kafka endpoint; connection string lives in Key Vault
kafka_options = {
    "kafka.bootstrap.servers": "<ns>.servicebus.windows.net:9093",
    "subscribe": "iot-telemetry",
    "kafka.security.protocol": "SASL_SSL",
    "kafka.sasl.mechanism": "PLAIN",
    "kafka.sasl.jaas.config": (
        'kafkashaded.org.apache.kafka.common.security.plain.'
        'PlainLoginModule required username="$ConnectionString' '
        f'password="{EVENTHUB_CONN_STR}";'
    ),
}

raw = (spark.readStream.format("kafka").options(**kafka_options).load()
 .select(F.from_json(F.col("value").cast("string"),
        "deviceId STRING, tempC DOUBLE, ts DOUBLE").alias("e"))
 .select("e.*").withColumn("event_time", F.from_unixtime("ts").cast("timestamp")))

agg = (raw.withWatermark("event_time", "10 minutes")
 .groupBy(F.window("event_time", "5 minutes"), "deviceId")
 .agg(F.avg("tempC").alias("avg_temp")))
```

```
(agg.writeStream.format("delta")
  .outputMode("append")
  .option("checkpointLocation", "abfss://checkpoints@<adls>.dfs.core.windows.net/iot")
  .toTable("iot_agg"))
```

- **Friday: Use Case & Architecture:** Design a pipeline to anonymize incoming healthcare HL7 messages in real-time using Databricks before streaming them into a Synapse workspace for analysis.

Resources: Spark Structured Streaming programming guide (<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>); Apache Spark project (<https://github.com/apache/spark>). **Self-check:**

- Micro-batch vs. continuous processing: what is the latency trade-off?
- Why does unbounded state kill a streaming job, and how do watermarks evict it?
- What must both sides of a stream-stream join declare for state to stay bounded?
- Why must you never reuse a checkpoint directory after changing a streaming query?
- Contrast checkpointing plus an idempotent Delta merge sink with at-least-once plus down-stream dedup.

Production Note: Practice checkpoint-location discipline during deployments: a checkpoint directory binds offsets, state, and the query plan together, so never reuse a checkpoint dir across query changes — a modified query starting from an old checkpoint can replay data, corrupt state, or fail to start. Version checkpoint paths per deployment (e.g., `.../ad-join/v3`) and start a changed query from a fresh location, backfilling from the source if needed.

Week 16: Business Intelligence, Power BI & Reverse ETL

Outcomes: build a DirectQuery Power BI dashboard, enforce Row-Level Security roles, write basic DAX measures, sync a curated warehouse segment back to a CRM via reverse ETL.

- **Monday:** Power BI Architecture: Desktop, Service, Datasets, and Dataflows.
- **Tuesday:** DirectQuery vs. Import mode vs. Dual mode.
- **Wednesday:** Row-Level Security (RLS) and DAX basics. Reverse ETL: a Synapse/ADF pipeline pushing curated segments (e.g., high-value churn-risk customers) from the warehouse into Dataverse/Dynamics 365 or a CRM API, including identity resolution (matching the same person across systems) and sync-frequency trade-offs (freshness vs. API cost and rate limits).
- **Thursday: Hands-on Project:** Connect Power BI Desktop to your Synapse Dedicated Pool using DirectQuery. Create a real-time dashboard monitoring sales, apply a filter, and set up an RLS role.
- **Friday: Use Case & Architecture:** Design the BI access layer for an enterprise where 500 analysts need to query centralized data, ensuring regional sales managers can only see data for their specific regions (RLS).

Resources: Power BI documentation (<https://learn.microsoft.com/power-bi/>); Power BI DirectQuery guidance (<https://learn.microsoft.com/power-bi/connect-data/desktop-directquery-about>). **Self-check:**

- When does DirectQuery beat Import mode?
- How does RLS restrict what a regional sales manager sees?
- What is reverse ETL, and why push warehouse segments back into a CRM?
- What does identity resolution reconcile before a reverse-ETL sync?
- What trade-off drives the choice of reverse-ETL sync frequency?

CAPSTONE PROJECT - Milestone 4: Real-Time Fraud Detection. Build a Python script simulating credit card swipes sent to Azure Event Hubs. Create an Azure Stream Analytics job that uses a Sliding Window to detect if a card is used in two different cities within 10 minutes, flags it, routes it to an Azure SQL Database, and displays the live fraud alerts in a streaming Power BI dashboard.

Acceptance Criteria

- End-to-end
- Architecture
- ASA
- At least
- out-of
- Monitor
- alert
- Monitor
- decision
- Security
- hand

Milestone 4 Capstone: Reference Solution Sketch

- **Architecture (one line):** Python producer → Event Hubs partitioned by `card_id` → ASA sliding-window fraud query → Azure SQL alerts table → Power BI streaming dashboard — ASA over Databricks because the rule is expressible in SQL and serverless SUs are cheaper at this volume.
- **Steps, in order:** (1) create the hub with enough partitions for peak throughput; (2) write the producer with a unique `eventId` per swipe; (3) author the ASA job with explicit late-arrival/out-of-order policies; (4) sink alerts to Azure SQL; (5) build the Power BI streaming dataset and dashboard; (6) add a watermark-delay alert in Azure Monitor.
- Hardest part — the fraud detection query itself:

```
SELECT card_id, COUNT(*) AS swipes,
       COUNT(DISTINCT city) AS cities
INTO fraud_alerts
FROM card_swipes TIMESTAMP BY event_ts
GROUP BY card_id, SlidingWindow(minute, 10)
HAVING COUNT(DISTINCT city) > 1
```

- **Pitfall:** default late-arrival policy silently drops offline-device events — set and document the tolerance window explicitly.
- **Pitfall:** partitioning by a low-cardinality key (e.g., merchant) skews load; partition by `card_id` so one card's events stay ordered.
- **Pitfall:** Power BI push datasets cap at 200k rows/hour — aggregate in ASA before the dashboard, never stream raw swipes to it.

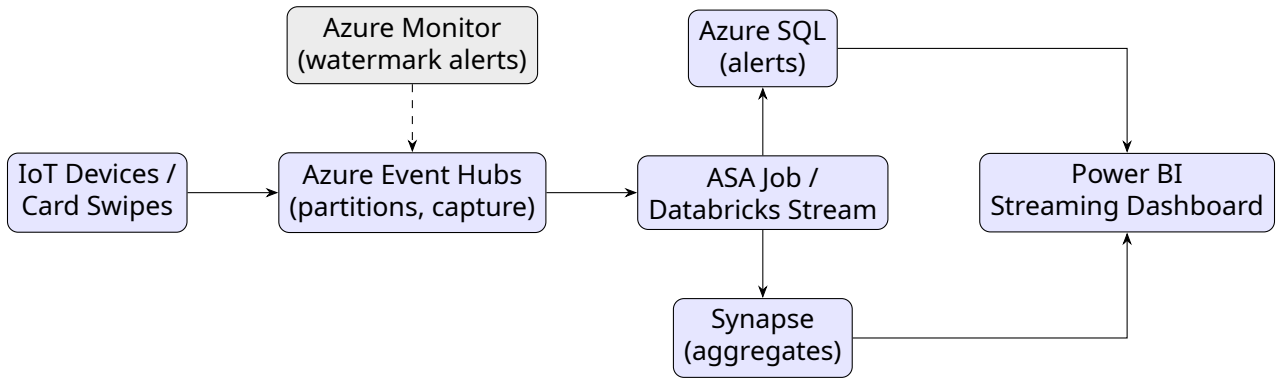


Figure 7: Milestone 4 reference architecture: streaming (IoT → Event Hubs → ASA/Databricks → Synapse → Power BI)

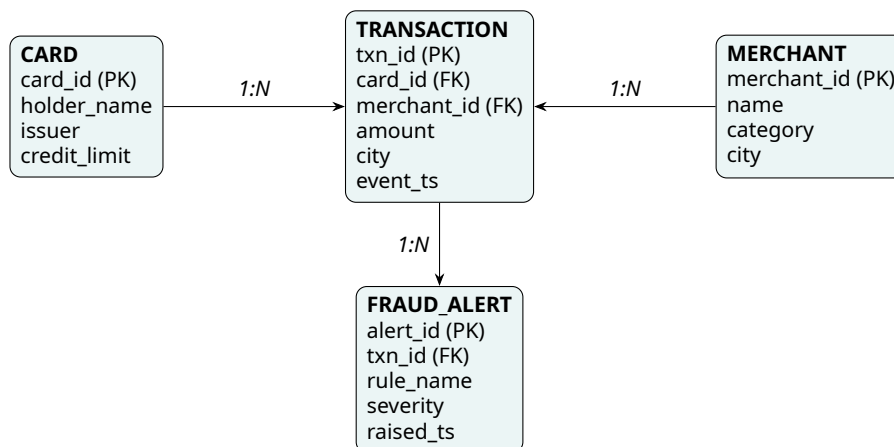


Figure 8: Capstone data model: real-time fraud detection

5 Milestone 5: Machine Learning Operationalization (Month 5)

Objective: Embed AI into data pipelines and manage the MLOps lifecycle with Azure ML.

Learning outcomes: score models in-database with T-SQL PREDICT, deploy managed AML endpoints, build AML pipelines with the Python SDK, detect data drift, integrate Cognitive Services into data pipelines.

Week 17: In-Database Machine Learning (Synapse PREDICT)

Outcomes: train a model with Synapse Spark, export it in ONNX format, score live rows with T-SQL PREDICT, keep training and scoring feature schemas aligned.

- **Monday:** Synapse Analytics Machine Learning integration.
- **Tuesday:** T-SQL PREDICT function to score data natively in Dedicated SQL Pools.
- **Wednesday:** Training models using Spark in Synapse and deploying to SQL Pools.
- **Thursday: Hands-on Project:** Use Spark in Synapse to train a logistic regression model on customer data to predict churn. Export the model, and run a T-SQL PREDICT query to score live data.
- **Friday: Use Case & Architecture:** Architect a system to automatically forecast daily inventory needs for 1,000 retail stores using purely SQL-based pipelines to minimize data movement.

Resources: Synapse PREDICT documentation (<https://learn.microsoft.com/azure/synapse-analytics/machine-learning/tutorial-sql-pool-model-scoring-predict>); Synapse ML library (<https://microsoft.github.io/SynapseML/>). **Self-check:**

- What does T-SQL PREDICT eliminate from the scoring path?
- When is in-database scoring preferable to a real-time endpoint?
- In what format must a model be stored for Synapse PREDICT?
- Why forecast inventory with purely SQL-based pipelines?
- What must be true of the training and scoring feature schemas?

Week 18: Azure Machine Learning (AML) Foundations

Outcomes: run an AutoML experiment on an ADLS dataset, deploy the best model to a managed online endpoint, test it with a REST payload, secure training-data access via Managed Identity.

- **Monday:** Azure Machine Learning Studio, Compute Instances, and Compute Clusters.
- **Tuesday:** Automated ML (AutoML) vs. Designer (Drag-and-drop) vs. Custom Notebooks.
- **Wednesday:** AML Managed Endpoints: Real-time and Batch Inference.
- **Thursday: Hands-on Project:** Use Azure AutoML to train a classification model on a cleaned ADLS dataset. Deploy the best model to a Managed Online Endpoint and send a test REST payload.

- **Friday: Use Case & Architecture:** Design a workflow for data scientists to train custom PyTorch models on GPU compute clusters while securely accessing training data stored in a governed ADLS data lake.

Resources: Azure Machine Learning documentation (<https://learn.microsoft.com/azure/machine-learning>)
 Azure ML examples repository (<https://github.com/Azure/azureml-examples>). **Self-check:**

- When would you choose AutoML over a custom training notebook?
- Real-time vs. batch managed endpoints: which fits nightly scoring?
- Why use AML compute clusters instead of compute instances for training?
- How does AML securely access governed ADLS training data?
- What does model registration give you beyond saving a file?

Week 19: MLOps & Azure ML Pipelines

Outcomes: build an AML pipeline from versioned components, register features in the managed feature store, detect data drift, wire a drift alert to an automated retraining run.

- **Monday:** Azure ML Managed Feature Store concepts.
- **Tuesday:** Azure ML Pipelines (Components and Environments) for MLOps orchestration.
- **Wednesday:** Data Drift detection and Model Monitoring in AML.
- **Thursday: Hands-on Project:** Build an Azure ML Pipeline (Preprocess → Train → Evaluate → Register). Execute the pipeline using the Python SDK from a Notebook.
- **Friday: Use Case & Architecture:** Architect an MLOps lifecycle that detects data drift in a production recommendation engine and automatically triggers a retraining pipeline using Azure Monitor/Event Grid.

Resources: AML pipelines documentation (<https://learn.microsoft.com/azure/machine-learning/concept-ml-pipelines>); Azure ML examples repository (<https://github.com/Azure/azureml-examples>).
Self-check:

- What problem does a managed feature store solve across teams?
- What should trigger a retraining pipeline in a mature MLOps setup?
- What is data drift, and how does AML detect it?
- Why are AML pipeline components versioned?
- What should gate model promotion to production?

Week 20: Unstructured Data & Azure Cognitive Services

Outcomes: extract key-value pairs with Document Intelligence, trigger document processing with a Function on blob upload, index extracted metadata in Cognitive Search, govern OCR-exposed PII.

- **Monday:** Azure Computer Vision and Azure Form Recognizer (Document Intelligence).
- **Tuesday:** Azure Cognitive Service for Language (NLP) and Translator.
- **Wednesday:** Integrating Cognitive Services within Azure Data Factory pipelines.

- **Thursday: Hands-on Project:** Write an Azure Function triggered by an ADLS Gen2 upload that sends a scanned PDF invoice to Azure Form Recognizer, extracts key-value pairs, and saves them as JSON.
- **Friday: Use Case & Architecture:** Design a scalable pipeline to process terabytes of uploaded user images, extract text (OCR), tag objects, and make the metadata searchable using Azure Cognitive Search.

Resources: Azure AI Document Intelligence (<https://learn.microsoft.com/azure/ai-services/document-intelligence/>); Azure Cognitive Search documentation (<https://learn.microsoft.com/azure/search/>). **Self-check:**

- Which service extracts key-value pairs from scanned invoices?
- Why trigger document processing with a Function on blob upload?
- How do you make extracted document metadata searchable?
- When is the Translator service needed inside a data pipeline?
- What PII risks arise when OCR-ing user-uploaded documents?

CAPSTONE PROJECT - Milestone 5: End-to-End MLOps Pipeline. Use Azure Data Factory to orchestrate a pipeline that extracts features via Synapse, pushes them to Azure ML, triggers an Azure ML Pipeline to train a custom model, registers the model in the AML Model Registry, and sets up a batch endpoint to score new data nightly.

Acceptance Criteria

- End-to-end pipeline → retrain
- Architecture → drift
- At least one output
- Monitor shot
- Monitor decision
- Security data

Milestone 5 Capstone: Reference Solution Sketch

- **Architecture (one line):** ADF orchestrates Synapse feature extraction → AML pipeline (preprocess → train → evaluate) → metric-gated registration in the Model Registry → nightly batch endpoint, with an Event Grid drift alert triggering retraining — batch endpoints because scoring is offline and high-volume.
- **Steps, in order:** (1) register the ADLS datastore using the workspace Managed Identity; (2) define pipeline components with pinned environments; (3) add an evaluation gate (e.g., $AUC \geq 0.80$) before registration; (4) create the batch endpoint and schedule it nightly; (5) configure the drift monitor on production inputs; (6) wire Event Grid → retraining pipeline.
- Hardest part — the AML pipeline definition with the SDK:

```

from azure.ai.ml import dsl, Input

@dsl.pipeline()
def churn_pipeline(raw: Input):
    prep = preprocess_component(input_data=raw)
    train = train_component(train_data=prep.outputs.clean)
    eval_ = evaluate_component(model=train.outputs.model)
    return {"model": train.outputs.model}

ml_client.jobs.create_or_update(
    churn_pipeline(raw=Input(path="azureml:features:1"))

```

- **Pitfall:** training/serving feature skew — generate both from the same versioned feature set, not ad-hoc SQL.
- **Pitfall:** promoting every trained model — gate registration on the metric threshold plus human approval.
- **Pitfall:** leaving AML compute running idle — use autoscaling clusters with a 0-node minimum.

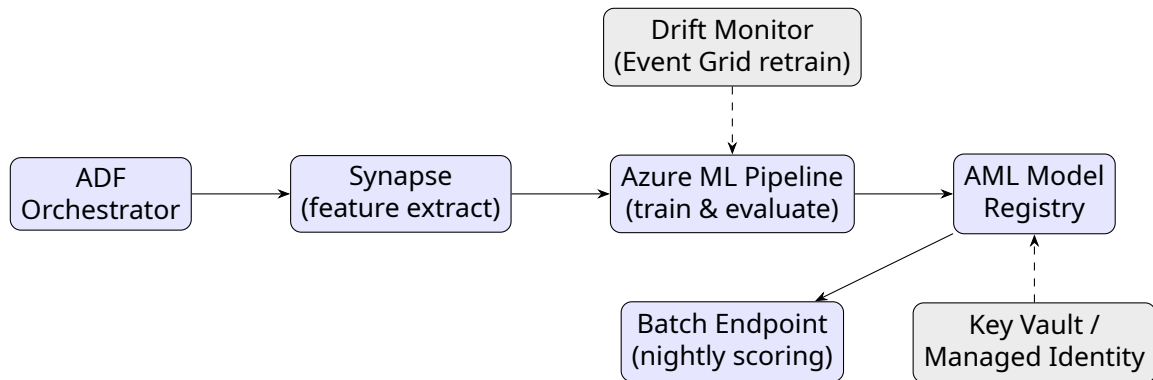


Figure 9: Milestone 5 reference architecture: MLOps with Azure ML (ADF → Synapse → AML → registry → batch endpoint)

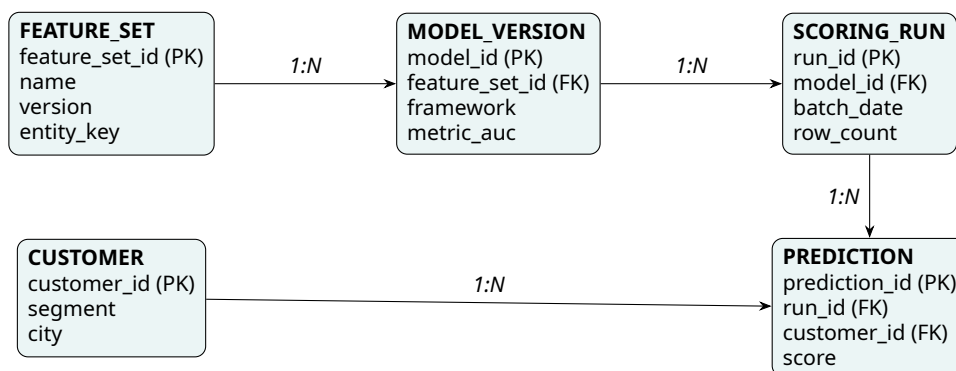


Figure 10: Capstone data model: end-to-end MLOps pipeline

6 Milestone 6: Security, Governance & Exam Prep (Month 6)

Objective: Secure the perimeter, implement Data Mesh, and prepare for DP-700 (Implementing Data Engineering Solutions Using Microsoft Fabric) — note: DP-203 was retired in 2025 and DP-700 is its successor for Azure data engineering.

Learning outcomes: enforce zero-trust networking with Private Endpoints, automate secret rotation with Key Vault, design a Purview-governed Data Mesh, provision data platforms with Terraform/Bicep, run dbt and CI/CD pipelines for data.

Week 21: Security, Azure AD & Networking

Outcomes: deploy a Private Endpoint and disable public access on ADLS, replace service-principal secrets with Managed Identities, automate Key Vault rotation, verify private-IP-only access from inside a VNet.

- **Monday:** Microsoft Entra ID (Azure AD), Service Principals, and Managed Identities.
- **Tuesday:** VNet Integration: Private Endpoints, Service Endpoints, and Azure Bastion.
- **Wednesday:** Azure Key Vault for secrets and Customer-Managed Keys (CMK).
- **Thursday: Hands-on Project:** Create a Private Endpoint for an ADLS Gen2 account. Disable public access. Use a VM within the same VNet to securely access the storage account via private IP.
- **Friday: Use Case & Architecture:** Design a highly secure data perimeter for a healthcare provider ensuring no data exfiltration can occur, utilizing VNet Data Exfiltration protection in Synapse.

Resources: Microsoft Entra ID documentation (<https://learn.microsoft.com/entra/identity/>); Azure Private Link documentation (<https://learn.microsoft.com/azure/private-link/>). **Self-check:**

- Why prefer Managed Identities over service principal client secrets?
- Private Endpoint vs. Service Endpoint: what is the key difference?
- What does the Key Vault `SecretNearExpiry` event enable?
- How does disabling public access change the access pattern?
- What does VNet data-exfiltration protection in Synapse guard against?

Production Note: Enable Key Vault secret rotation with rotation policies and Event Grid notifications (`SecretNearExpiry`) so certificates and connection strings rotate automatically. Pipeline connections should reference Key Vault secrets via Managed Identity — a rotated secret must never require a code change.

Week 22: Advanced Data Governance (Purview)

Outcomes: define a custom Purview classification rule, gate pipeline loads with dbt tests or GE checkpoints, propagate sensitivity labels to Synapse and Power BI, design centralized governance for a decentralized Data Mesh.

- **Monday:** Microsoft Purview: Data map, classifications, and sensitivity labels.

- **Tuesday:** Data policies and access provisioning via Purview.
- **Wednesday:** Integrating Purview with Azure Synapse and Databricks for holistic lineage; automated data-quality testing with Great Expectations or dbt tests running inside Databricks/ADF pipelines.
- **Thursday: Hands-on Project:** Define a custom classification rule in Microsoft Purview for a proprietary employee ID format. Scan a data lake to automatically discover and tag this data.
- **Friday: Use Case & Architecture:** Architect a Data Mesh for a global conglomerate with 5 independent domains, ensuring centralized governance via Microsoft Purview but decentralized data engineering.

Resources: Microsoft Purview data lineage (<https://learn.microsoft.com/azure/purview/concept-data-lineage>); Great Expectations project (https://github.com/great-expectations/great_expectations); dbt Core (<https://github.com/dbt-labs/dbt-core>). **Self-check:**

- How do sensitivity labels flow from Purview to Synapse and Power BI?
- Why run dbt tests or GE checkpoints before the load step?
- What does Purview lineage show when a report suddenly breaks?
- In a Data Mesh, what stays centralized and what is decentralized?
- Why define custom classifications for proprietary ID formats?

Production Note: *Fail pipelines on data-quality rule violations: run Great Expectations checkpoints or `dbt test` as a gating step before the load activity in ADF/Databricks. Route failed batches to a quarantine container, emit a metric to Azure Monitor, and alert on-call — silently loading bad data is worse than a delayed pipeline.*

Week 23: Compliance & Data Privacy

Outcomes: apply Dynamic Data Masking and verify masked views, distinguish Always Encrypted from TDE, execute a GDPR erasure via crypto-shredding across lake, warehouse, and streaming copies, handle legal-hold exceptions without breaking the audit trail.

- **Monday:** Dynamic Data Masking (DDM) in Azure SQL and Synapse.
- **Tuesday:** Always Encrypted and Transparent Data Encryption (TDE).
- **Wednesday:** Azure Policy and Microsoft Defender for Cloud (Auditing and Alerts). Legal-hold interaction: how Azure Policy effects, resource locks, and Purview retention labels can block or defer an erasure, and how to flag a data subject under legal hold so erasure propagation is suspended while the audit trail (who requested, who deferred, when it resumed) is preserved.
- **Thursday: Hands-on Project:** Implement Dynamic Data Masking on an Azure SQL Database table containing PII. Create a read-only user and verify that they see masked data (e.g., xxx-xxxx) for emails. Then run a GDPR right-to-erasure drill based on crypto-shredding: delete one subject's per-subject key from Azure Key Vault and prove the data is unreadable in every copy — raw and curated ADLS Gen2 zones, Synapse Dedicated and Serverless pool tables, events still inside the Event Hubs retention window, downstream derived tables, and Redis/query caches — documenting how soft-delete and backup windows expire without ever holding a decryptable copy.

- **Friday: Use Case & Architecture:** Design a pipeline that ingests third-party customer data, automatically identifies PII, tokenizes it using Azure Key Vault secrets, and stores the safe version in Synapse.

Resources: Azure SQL Dynamic Data Masking (<https://learn.microsoft.com/azure/azure-sql/database/dynamic-data-masking-overview>); Microsoft Defender for Cloud (<https://learn.microsoft.com/azure/defender-for-cloud/>). **Self-check:**

- Who still sees unmasked values when Dynamic Data Masking is enabled?
- Always Encrypted vs. TDE: which protects data from database admins?
- What does Defender for Cloud add on top of auditing?
- Why tokenize PII before loading it into Synapse?
- Where do tokenization keys belong, and how are they accessed?

Production Note: *Naive DELETE is not erasure: soft-delete windows, blob versioning, automated backups, Delta time-travel history, and Event Hubs retention all keep copies of the row after the DELETE commits. Design for crypto-shredding — one Key Vault key per data subject, so deleting the key renders every copy (including backups and time-travel history) permanently unreadable — or maintain a documented per-store purge/replay runbook whose SLAs match the longest retention window. Subjects under legal hold must be excluded from propagation and reprocessed when the hold lifts.*

Week 24: Production DataOps (IaC, dbt & CI/CD) & Certification Prep

Outcomes: provision ADLS and ADF with Terraform/Bicep, gate dbt changes with GitHub Actions, deploy a schema change with the expand-contract pattern, plan a DP-700 revision schedule from mock-exam results.

- **Monday:** Infrastructure as Code with Terraform (azurerm provider) for ADLS Gen2, Synapse, and Data Factory; Bicep as the Azure-native alternative.
- **Tuesday:** dbt with Synapse/Fabric: models, tests, and documentation.
- **Wednesday:** CI/CD for data pipelines with GitHub Actions or Azure DevOps: ARM/Bicep deployment, ADF publishing via the npm ADFUtilities package concept, and environment promotion (dev → test → prod). Zero-downtime schema evolution with expand-contract, coordinated with the dbt/GitHub Actions pipeline: (1) expand — add the new nullable column alongside the old one; (2) migrate — dual-write new writes and backfill history (dbt incremental run or ADF copy); (3) switch reads behind a backward-compatible view shim so consumers keep one stable interface; (4) contract — drop the old column only after Purview lineage and the dbt `ref()` graph prove no consumer still reads it, with each step shipped as a separate ordered release.
- **Thursday: Hands-on Project:** Create a GitHub repository containing Terraform code that provisions an ADLS Gen2 account and an ADF instance, plus a dbt project with 3 models and 5 tests. Add a GitHub Actions workflow that validates Terraform and runs dbt tests on every pull request.

```
terraform {
  required_providers {
    azurerm = { source = "hashicorp/azurerm", version = "~> 4.0" }
  }
}
```

```

provider "azurerm" { features {} }

resource "azurerm_storage_account" "adls" {
  name                = "stadlsdemo001"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  account_tier        = "Standard"
  account_replication_type = "LRS"
  is_hns_enabled      = true # ADLS Gen2 hierarchical namespace
}

resource "azurerm_data_factory" "adf" {
  name                = "adf-demo-001"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  identity { type = "SystemAssigned" }
}

```

- **Friday:** Compressed DP-700 prep: review exam domains (Fabric lakehouses, ingestion & transformation, monitoring & optimization), identify weak areas, and schedule both full-length mock exams for the weekend following the program.

Resources: DP-700 certification overview (<https://learn.microsoft.com/credentials/certifications/data-engineer/>); Terraform azurerm provider docs (<https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs>); dbt Core (<https://github.com/dbt-labs/dbt-core>). **Self-check:**

- Why keep Terraform state in an Azure Storage backend with locking?
- What does `dbt test` add on top of dbt models?
- Why run `terraform plan` on PRs but apply only after review?
- What does environment promotion (dev → test → prod) protect against?
- Which DP-700 domains should you weight most in your revision, and why?

Production Note: *Failure story: a team deployed the contract step first — a GitHub Actions release dropped `customer_email` while the dbt model and the Power BI dataset still referenced it; dashboards failed at 09:00 and the fix needed a hot rollback plus a full dbt re-run. Expand-contract exists so every intermediate state is valid: never drop a column until a dependency check (Purview lineage plus the dbt `ref()` graph) proves no consumer reads it, and ship the view shim in the same PR as the column change so reads never break between releases.*

Production
and never
— a destru

- End- (Terr
- Arch subs
- At lea load
- Mon func
- Mon ume
- Secu Iden

CAPSTONE PROJECT - Milestone 6: Secure, Governed Data Mesh. Build a multi-subscription architecture. Subscription A ingests streaming PII data and uses Azure Functions to tokenize it. Subscription B (Analytics) hosts Synapse. Subscription C (Governance) uses Microsoft Purview and Private Endpoints to lock down all traffic, ensuring a zero-trust network boundary.

Milestone 6 Capstone: Reference Solution Sketch

- **Architecture (one line):** three subscriptions — A ingests and tokenizes PII (Event Hubs + Functions + Key Vault), B hosts Synapse/ADLS analytics, C runs Purview governance — peered VNets with Private Endpoints everywhere and zero public network access, all deployed by Terraform/Bicep modules.
- **Steps, in order:** (1) write a Terraform module per subscription with an Azure Storage state backend; (2) peer VNets and create private DNS zones; (3) deploy the tokenization Function with Key Vault references; (4) deploy Synapse with public access disabled; (5) register sources in Purview and scan; (6) gate the tokenized load with dbt tests in GitHub Actions; (7) prove zero public reachability from outside the VNets.
- Hardest part — the private endpoint with DNS (endpoints without DNS resolve publicly and silently fail):

```
resource "azurerm_private_endpoint" "adls" {
  name            = "pep-adls"
  subnet_id       = azurerm_subnet.data.id
  private_service_connection {
    private_connection_resource_id = azurerm_storage_account.adls.id
    subresource_names              = ["dfs"]
  }
  private_dns_zone_group {
    private_dns_zone_ids = [azurerm_private_dns_zone.dfs.id]
  }
}
```

- **Pitfall:** creating the Private Endpoint but not the DNS zone — name resolution still returns the public IP and traffic fails or leaks.
- **Pitfall:** `public_network_access_enabled` defaults to true on several resources — set it to false explicitly in code.
- **Pitfall:** cross-subscription RBAC sprawl — centralize role assignments in the governance subscription and audit them monthly.

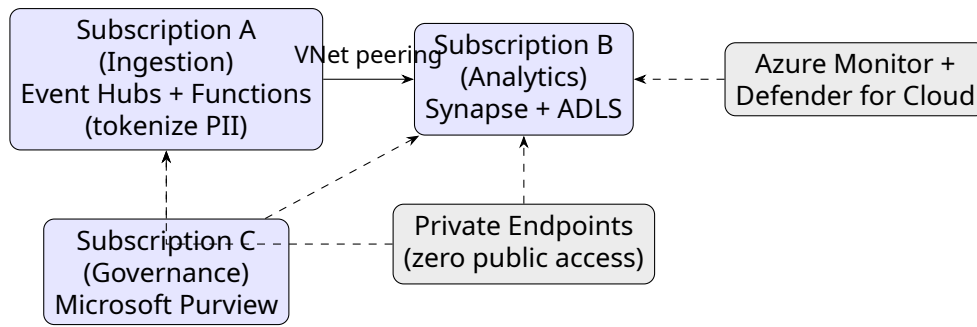


Figure 11: Milestone 6 reference architecture: multi-subscription zero-trust Data Mesh

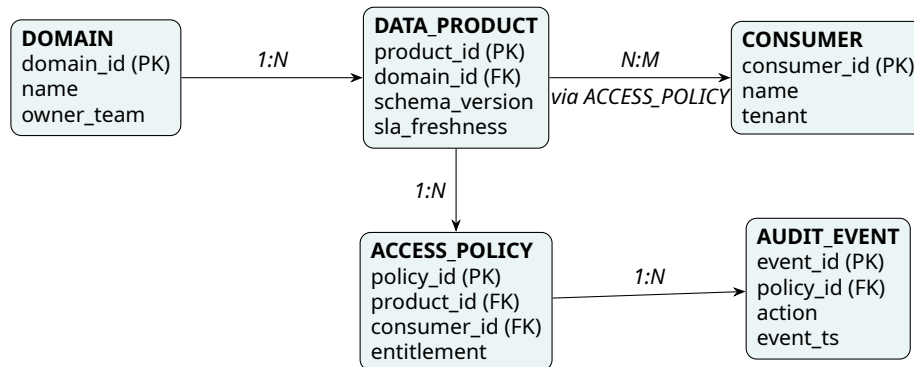


Figure 12: Capstone data model: governed data mesh

7 Appendix A: Glossary

ADLS Gen2	Azure Data Lake Storage Gen2 — Blob Storage with a hierarchical namespace for analytics.
ADF	Azure Data Factory — managed ETL/ELT orchestration service with pipelines and Mapping Data Flows.
ASA	Azure Stream Analytics — serverless real-time event processing with a SQL-like query language.
Bicep	Azure-native declarative Infrastructure-as-Code language that transpiles to ARM templates.
CDC	Change Data Capture — streaming of insert/update/delete changes from a source database.
CETAS	CREATE EXTERNAL TABLE AS SELECT — serverless SQL pattern to materialize query results to the lake.
Cosmos DB	Globally distributed multi-model NoSQL database billed in Request Units.
cDWU	Compute Data Warehouse Unit — the compute sizing metric of Synapse Dedicated SQL Pools.
dbt	SQL-based transformation framework with models, tests, and documentation as code.
Delta Lake	Open table format adding ACID transactions and time travel to data lakes.

DMS	Azure Database Migration Service — orchestrates schema and data migration with optional CDC.
DP-700	Implementing Data Engineering Solutions Using Microsoft Fabric — the Azure data engineering certification succeeding DP-203.
Event Hubs Capture	Automatic batched delivery of Event Hubs streams to ADLS Gen2 or Blob Storage.
Integration Runtime	The ADF compute used to execute activities (Azure, Self-Hosted, or Azure-SSIS).
Key Vault	Managed store for secrets, keys, and certificates with rotation and access policies.
Lakehouse	Architecture combining data lake storage with warehouse-style ACID tables (Delta Lake).
Managed Identity	Entra ID identity managed by Azure, eliminating stored credentials in code.
Mapping Data Flow	ADF's visual, code-free transformation engine executed on managed Spark.
Private Endpoint	Network interface giving a resource a private IP inside your VNet.
Purview	Microsoft's unified data governance, catalog, lineage, and classification service.
Request Unit (RU)	Cosmos DB throughput currency; every operation consumes RUs.
Structured Streaming	Spark's micro-batch/continuous stream processing API over DataFrames.
Synapse Analytics	Azure's analytics service unifying Dedicated/Serverless SQL pools and Spark.
Terraform azurerm	HashiCorp provider for provisioning Azure resources declaratively.
Watermark	Stream-processing tolerance defining how long late events are still accepted.

8 Appendix B: Assessment & Grading Model

Each monthly capstone is graded on the following weighted rubric:

Category	Weight
Correctness (pipeline works end-to-end)	30%
Reliability & tests (DQ tests, retries, idempotency)	20%
Security (least privilege, secrets, PII)	15%
Cost awareness (estimate + optimizations)	15%
Documentation (README, diagram, runbook)	10%
Demo (walkthrough of the working system)	10%

A passing capstone requires $\geq 70\%$ overall and no zero in any category. The final readiness score combines the six capstone grades (60%) with the two DP-700 mock exams taken after Week 24 (40%): score $\geq 75\%$ to consider yourself certification-ready.

9 Appendix C: Interview Preparation

Ten questions this program prepares you to answer, with the weeks that teach them:

1. When would you choose Cosmos DB over Azure SQL, and how do partition keys drive that decision? (Weeks 2–3)
2. Explain Synapse Dedicated vs. Serverless SQL Pools and how you would control cost. (Weeks 5–7)
3. How does Delta Lake provide ACID guarantees on ADLS Gen2? (Weeks 7, 9)
4. Walk me through an event-driven ELT pipeline from file upload to warehouse load. (Weeks 10–12)
5. How do you design Event Hubs partitioning and guarantee idempotent consumption? (Weeks 13–14)
6. Compare Azure Stream Analytics and Spark Structured Streaming for a real-time use case. (Weeks 14–15)
7. How do you implement Row-Level Security for 500 analysts in Power BI? (Week 16)
8. Describe an MLOps lifecycle: drift detection, retraining, and gated model promotion. (Weeks 18–19)
9. How would you build a zero-trust data perimeter so no service is publicly reachable? (Weeks 21, 24)
10. How do you enforce data quality and governance at scale across a Data Mesh? (Weeks 8, 22)

10 Appendix D: Self-Check Answer Key

Terse answers to the weekly Self-check questions. Attempt all five before reading them.

Week 1:

1. ADLS Gen2 adds a hierarchical namespace and POSIX-style ACLs over Blob Storage.
2. Rarely accessed data; cheapest storage but hours-long rehydration latency.
3. Automatically tiers or deletes blobs based on age-based rules.
4. They retain deleted/overwritten blobs so prior versions can be restored.
5. AzCopy over the network; Data Box for multi-TB offline transfer.

Week 2:

1. vCore — you pick hardware, cores, and storage directly; DTU is a bundled unit.
2. Managed, automatic failover of a group of databases to another region.
3. Avoids overwriting the live database; allows verification before cutover.

4. It auto-pauses when idle, billing only storage during downtime.
5. Geo-replication is an async DR secondary; read scale-out serves reads in-region.

Week 3:

1. RU prices every operation; cross-partition fan-out multiplies RU consumption.
2. Strong consistency.
3. Per-session read-your-writes without strong-consistency latency and cost.
4. Ordered changes within a partition key; no cross-partition ordering.
5. Skewed keys create hot partitions that throttle; even keys spread RU load.

Week 4:

1. Read cache first, populate on miss, invalidate on write — for read-heavy data.
2. Integrated cache lives inside Cosmos DB; Redis is a standalone general-purpose cache.
3. Centralized throttling, authentication, quotas, and monitoring for ingestion APIs.
4. They go stale; set TTLs or explicit invalidation on writes.
5. Autoscale the API tier, cache hot keys, and throttle via APIM policies.

Week 5:

1. The grain defines what one row means; dimensions and measures follow from it.
2. Accumulating snapshot — one row per order, updated at each milestone.
3. When historical attribute values matter, e.g., customer segment at sale time.
4. Shared dimensions let marts drill across and join consistently enterprise-wide.
5. Low-volume, spiky lake queries — you pay only per TB scanned.

Week 6:

1. Columnar compression and batch execution make big scans and aggregates fastest.
2. Co-locates matching join keys on the same node, avoiding data shuffles.
3. Small dimension tables joined frequently — a full copy on every node.
4. Workload groups with importance and resource allocations.
5. Repeated aggregations over huge fact tables — precomputed results answer instantly.

Week 7:

1. Per TB scanned; reduce with Parquet, partition pruning, and selective columns.
2. Exposing lake files as views/external tables without copying into a warehouse.
3. A transaction log providing ACID commits, time travel, and schema enforcement.
4. Materializes a query result into lake files as a new external table.
5. Columnar, compressed, schema-typed — far cheaper to scan than CSV.

Week 8:

1. Schemas, classifications (e.g., PII), and metadata across registered sources.
2. It shows which downstream reports and pipelines a change will affect.
3. Governed, revocable, auditable sharing instead of unmanaged file copies.
4. A governed space to join parties' data; it must never expose raw PII.

5. Purview scan rule sets with built-in or custom classification patterns.

Week 9:

1. They terminate after the job, costing 2-3× less per DBU.
2. The number of parallel tasks and the width of shuffles.
3. Declarative pipelines with data-quality expectations on top of Delta.
4. When it is reused downstream multiple times and fits in cluster memory.
5. A workspace filesystem abstraction; production data belongs in mounted ADLS.

Week 10:

1. Linked Service stores the connection; Dataset points at the data within it.
2. For code-free analyst-built transformations; Spark wins for complex custom logic.
3. Sources sit on-prem or in a VNet unreachable by the Azure IR.
4. The pipeline fails — the default dependency is on-success only.
5. Global parameters plus ARM/Bicep templates swapped per environment at deploy time.

Week 11:

1. Fail fast and quarantine bad payloads before they poison the lake.
2. Non-breaking — consumers ignoring unknown fields keep working.
3. The producer owns it, versioned and negotiated with consumers.
4. Evolve schemas safely, running old and new versions in parallel during migration.
5. CDC streams inserts/updates/deletes continuously; full extracts reload everything nightly.

Week 12:

1. Until loops until a condition holds; ForEach iterates a known collection.
2. Running existing Airflow DAGs without managing Airflow infrastructure.
3. Decouples alerting and enables rich actions (Teams, tickets) beyond email.
4. Retries with backoff, idempotent activities, and restart-from-failure design.
5. Pipelines react to events (file arrival) instead of polling on a schedule.

Week 13:

1. Service Bus for per-message enterprise semantics; Event Hubs for high-throughput streams.
2. One consumer per partition per group — parallelism cannot exceed partition count.
3. Automatic batched landing of stream data into ADLS Gen2.
4. It enables idempotent deduplication on retries and at-least-once replays.
5. Independent read offsets so multiple apps consume the same stream separately.

Week 14:

1. Tumbling: fixed non-overlapping windows; hopping: fixed windows overlapping by hop size.
2. How long late events are still accepted into their original window.
3. Adjusted to the boundary or dropped, per the configured policy.
4. Both produce zero output rows; only watermark/input metrics distinguish them.

5. Widen late-arrival tolerance and add a batch correction/reconciliation layer.

Week 15:

1. Micro-batch: seconds latency, high throughput; continuous: milliseconds, less mature.
2. State grows until executors run out of memory; watermarks evict state older than the watermark.
3. Watermarks on both sides plus a time-range join condition bounding how long matches are awaited.
4. A checkpoint binds offsets and state to the old query plan — a changed query may replay or corrupt state.
5. Exactly-once: checkpointing plus idempotent Delta merge; the alternative writes at-least-once and dedupes downstream by event key.

Week 16:

1. Near-real-time freshness needs or models too large for import memory.
2. Role membership filters rows so each manager sees only their region.
3. Syncing curated warehouse segments back into operational tools (CRM, marketing).
4. Matching the same person across systems via keys, emails, or fuzzy rules.
5. Freshness vs. API cost and rate limits — pick the slowest acceptable cadence.

Week 17:

1. Moving data to an external scoring service — prediction happens where data lives.
2. High-volume batch scoring inside existing SQL pipelines.
3. An ONNX model serialized and stored in the pool (varbinary/hex string).
4. Minimizes data movement, latency, and extra services for simple forecasts.
5. Identical feature order and types, or scoring fails or silently misaligns.

Week 18:

1. Fast baselines when ML expertise or time is limited.
2. The batch endpoint — scheduled, high-volume offline scoring.
3. Autoscaling, job queuing, and pay-per-use shared across a team.
4. Via the workspace Managed Identity accessing a governed ADLS datastore.
5. Versioning, lineage, tags, and a governed handoff to deployment.

Week 19:

1. Consistent, versioned features shared between training and serving.
2. Drift alerts, schedules, or new-data arrival events via Event Grid.
3. Statistical change in input distributions vs. training; AML compares datasets.
4. For reproducible reruns and safe upgrades of individual pipeline steps.
5. Metric thresholds plus human approval before production deployment.

Week 20:

1. Azure AI Document Intelligence (formerly Form Recognizer).

2. Event-driven serverless scaling with per-file processing and no idle cost.
3. Index extracted fields into Azure Cognitive Search via an indexer/skillset.
4. When multilingual content must be normalized before downstream NLP.
5. OCR surfaces PII from images; govern storage, access, and retention of output.

Week 21:

1. No secrets to store, leak, or rotate — Azure manages the identity lifecycle.
2. A Private Endpoint gives the resource a private IP inside your VNet.
3. Event Grid automation triggering rotation pipelines before secrets expire.
4. Access only via VNet/VPN/ExpressRoute — the public internet path is removed.
5. Blocking egress of workspace data to unauthorized external locations.

Week 22:

1. Labels propagate from Purview scans through to Synapse and Power BI artifacts.
2. Bad batches get quarantined before reaching curated tables and consumers.
3. Which upstream table or column change broke which downstream report.
4. Central: governance and catalog (Purview); decentralized: domain-owned data products.
5. Built-in classifiers miss proprietary formats like internal employee IDs.

Week 23:

1. Users granted the UNMASK permission; everyone else sees masked values.
2. Always Encrypted — keys stay client-side, invisible to database admins.
3. Threat detection, vulnerability alerts, and regulatory compliance scoring.
4. It limits PII exposure; only tokenized surrogates reach the warehouse.
5. Azure Key Vault, accessed via Managed Identity with rotation policies.

Week 24:

1. Shared, lease-locked state prevents concurrent corrupting applies across a team.
2. Assertions (unique, not-null, accepted values) gating model correctness.
3. Plan is read-only review; apply can destroy resources — gate it.
4. Catching environment-specific issues before production data is touched.
5. Your weakest of: lakehouses, ingestion/transformation, monitoring/optimization.

11 Appendix E: Datasets & Project Data

Public datasets mapped to the monthly capstones, plus guidance for synthetic data:

- **NYC Taxi & Limousine trip records** (Azure Open Datasets, yellow/green taxis; <https://learn.microsoft.com/azure/open-datasets/dataset-taxi-yellow>) — stable, Azure-hosted, billions of rows available. Use for Milestones 2–3: Week 5 COPY lab (sample ~10M rows), star-schema modeling, and the batch ELT capstone.
- **NOAA Global Historical Climatology Network (GHCN)** (Azure Open Datasets; <https://learn.microsoft.com/azure/open-datasets/dataset-noaa-ghcn>) — global daily weather observations. Use for Milestone 4 streaming/time-series patterns and Milestone 5 forecasting features.

- **Olist Brazilian e-commerce dataset** (~100k orders; Kaggle and GitHub mirrors, e.g. <https://github.com/olist/work-at-olist-data>) — realistic marketplace schema (customers, orders, items, payments, reviews). Use for the Milestone 1 backend and Milestone 6 data-mesh capstones.
- **Synthetic data:** Faker (Python) for customers/orders/dimensions; custom generators for event streams. Expected volumes — Milestone 1: $\sim 10^4$ rows; Milestones 2–3: $\sim 10^7$ rows of Parquet; Milestone 4: 1–5 events/sec sustained streams; Milestone 5: $\sim 10^5$ labeled rows.

12 Appendix F: Portfolio & Presentation Guide

Each capstone is portfolio material. Publish it on GitHub as follows.

README structure: (1) one-paragraph problem statement; (2) the architecture diagram first, before any code; (3) setup instructions runnable from a clean environment; (4) screenshots of the pipeline run, dashboard, and alerts; (5) a monthly cost estimate with the optimizations you chose; (6) lessons learned and what you would do differently.

Resume bullet templates (two per milestone):

- *M1:* “Built a multi-tier Azure data backend (Azure SQL Serverless, Cosmos DB, ADLS Gen2) with Change Feed event propagation, cutting reporting latency from hours to minutes.” / “Designed lifecycle-tiered ADLS storage with geo-redundancy, reducing monthly storage spend $\sim 40\%$ via Cool/Archive policies.”
- *M2:* “Built a Delta Lake lakehouse on ADLS Gen2 served by Synapse Serverless SQL views and cataloged in Microsoft Purview, avoiding fixed dedicated-warehouse cost.” / “Modeled a Kimball star schema with SCD Type 2 dimensions in Synapse; tuned distribution keys, improving benchmark JOIN time by $[N] \times$.”
- *M3:* “Automated event-driven ELT (Event Grid $\rightarrow$ ADF $\rightarrow$ Databricks $\rightarrow$ Synapse) with JSON data-contract validation quarantining non-conforming files.” / “Implemented CDC replication via Azure DMS with contract enforcement, achieving < 5 -minute data freshness for BI consumers.”
- *M4:* “Built real-time fraud detection with Event Hubs and Stream Analytics sliding windows, surfacing alerts on a live Power BI dashboard.” / “Engineered idempotent Event Hubs/Kafka consumers with checkpointing, eliminating duplicate alerts under replay.”
- *M5:* “Operationalized ML with an ADF-orchestrated Azure ML pipeline featuring drift-triggered retraining and nightly batch scoring.” / “Deployed AutoML models to managed endpoints behind Managed Identity with metric-gated promotion.”
- *M6:* “Provisioned a zero-trust, multi-subscription data mesh with Terraform/Bicep and Private Endpoints — zero public data surface.” / “Established data CI/CD: dbt tests gating warehouse loads, terraform plan on PRs, and dev/test/prod promotion.”

Talking about your project in interviews: lead with the business problem, then draw the architecture from memory, then discuss trade-offs you weighed (cost vs. latency, consistency vs. availability, freshness vs. API load). Quantify everything you can (rows, latency, cost). Interviewers trust candidates who can name one thing that broke and how they fixed it — keep one failure story per capstone ready.

13 6-Month Progress Tracking Timetable

Week	Primary Focus	Key Project / Capstone	Status
1	ADLS Gen2 & Storage	Storage Lifecycle & Hierarchical Namespace	☐
2	Azure SQL & PostgreSQL	Failover Simulation & PITR Restore Drill	☐
3	Azure Cosmos DB	Cosmos DB Partitioning & Global Writes	☐
4	Hybrid Ingestion & Caches	Cap 1: E-Commerce Multi-Tier Data	☐
5	Synapse Analytics Basics	COPY into Synapse Dedicated SQL Pool	☐
6	Synapse Optimization	Distribution Types Benchmark	☐
7	Synapse Serverless SQL	Querying ADLS via CETAS & Logical Views	☐
8	Microsoft Purview	Cap 2: Enterprise Data Lakehouse	☐
9	Azure Databricks	PySpark Aggregation on ADLS Gen2	☐
10	Azure Data Factory (ADF)	ADF Mapping Data Flows Visual ETL	☐
11	Azure DMS, CDC & Data Contracts	CDC Pipeline with Contract Validation	☐
12	ADF Orchestration	Cap 3: Automated Batch ELT	☐
13	Azure Event Hubs	IoT Producer/Consumer Scripting	☐
14	Azure Stream Analytics	ASA Tumbling Windows to Azure SQL	☐
15	Spark Structured Streaming	Databricks Streaming from Event Hubs	☐
16	Power BI & Reverse ETL	Cap 4: Real-Time Fraud Telemetry	☐
17	Synapse PREDICT	Churn Prediction T-SQL Models	☐
18	Azure ML Foundations	AutoML Deployment & REST Endpoint Test	☐
19	Azure ML Pipelines	AML Pipeline Execution via Notebook	☐
20	Cognitive Services (AI APIs)	Cap 5: End-to-End MLOps Pipeline	☐
21	Entra ID & Private Endpoints	ADLS Private Endpoint implementation	☐
22	Purview Governance	Custom Classifications & Scans	☐
23	Dynamic Data Masking	Azure SQL DDM on PII Columns	☐
24	DataOps & DP-700 Prep	Cap 6: Secure Data Mesh Design	☐

"Consistency is the architecture of mastery."

Best of luck on your 6-month journey to becoming a Master Azure Data Engineer!